Chapter 3

Creating a Data Science Lab of Your Own

IN THIS CHAPTER

This book helps you gain an understanding of data science using two different languages: Python and R. Of course, not everyone will want to learn both languages, and you're not required to do so. Still, you need to choose one of them and create a data science lab of your own so that you can follow along with the examples. Consequently, this chapter begins by examining various options you have for creating your lab — they're extensive because data science has become so popular, and so many people with different skill sets need data science to perform tasks. After you choose the options you want, you usually need to perform some amount of setup to create your lab. This chapter also helps you toward that end.

As part of performing some data science tasks, you also use a framework. In most cases, your language will require that you import either

libraries or *packages* containing add-on code that makes performing data science tasks easier. The add-ons reside within your application and become part of it. A *framework* is different because it controls your application environment, and your application resides within it. Frameworks and libraries aren't mutually exclusive, and you generally use them together to perform specific tasks. This chapter provides you with additional details about frameworks and helps you understand what they can provide.

An *Integrated Development Environment (IDE)* also makes the task of performing data science tasks easier by providing you with an editor and associated tools to write code, compile it as needed, and perform tasks such as testing and debugging. This book relies on Anaconda, which is a suite of various tools used to create data science code, including the Jupyter Notebook IDE. This chapter doesn't provide you with a complete Anaconda tutorial; that task might require an entire book by itself. However, you do gain enough knowledge to perform essential tasks to make your learning experience better.

The final part of this chapter discusses the downloadable source code for this book. You don't have to manually type all the source code you see, and in fact it can be detrimental to do so because you might have to deal with issues like typos. Using the downloadable source makes your job a lot easier and helps you spend your time learning about data science rather than debugging faulty code. Of course, after you learn a particular topic, it's a good idea to practice what you learn by typing the code manually. You can find instructions for obtaining the downloadable source in the book's Introduction. This chapter helps you work with the code after you have it downloaded.

REMEMBER You don't have to type the source code for this chapter manually. In fact, it's a lot easier if you use the downloadable source. The source code for this chapter appears in the `DSPD_0103_Sample.ipynb`

source code file for Python and the `DSPD_R_0103_Sample.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Considering the Analysis Platform Options

Your analysis platform defines how you interact with the various tools used to perform analysis with data science. It also determines any limitations or requirements needed to implement the platform successfully. The two common approaches today are to set up a desktop PC with the required tools or to rely on an online IDE instead. Both options come with issues that you must consider. For example, the PC option provides environmental flexibility and reliability, but at a cost. Meanwhile, an online IDE provides locational flexibility, often at a greatly reduced cost, if not free.

REMEMBER The chapter also considers the use of the Graphics Processing Unit (GPU), which may seem confusing because most people associate a GPU with drawing objects onscreen. Using one or more GPUs can greatly speed your analysis. The final section helps you understand why a GPU can be so important, especially when performing complex analysis or using techniques such as deep learning.

Using a desktop system

Desktop systems offer significant flexibility in configuring a data-analysis environment specifically suited to your needs. You have control over every aspect of a desktop system, which means that if you don't like how a system performs, you can change it so that the performance becomes acceptable. For example, you can often make a desktop system fly by adding a GPU, additional memory, a faster processor, or even a faster hard drive. However, all this flexibility costs something. You need to buy the various pieces of hardware you want to

make the system happen. So, you might have to balance cost against needs and consider how much performance you can afford.

Along with system control, you must also consider issues like data control. A desktop system can maintain data on the local system, which can help you meet privacy, governmental, and other requirements. The consistent access provided by a local system can also make a difference. Of course, when using a desktop system, you must consider backup and data redundancy needs. You don't want a local lightning storm to wipe out all your hard work.

TIP A desktop system also comes with various human environment benefits. You don't have to deal with using a small screen for seeing output from a complex analysis as you might when working with a tablet. In fact, you can spread the output across as many screens as your system will support — two, in most cases, even with a low-end system. You also have aspects like better keyboard support and so on to consider. These comfort features may not seem important when performing analysis, but they can become essential when dealing with a huge amount of data over the long hours needed to work with it.

The most important consideration when working with desktop systems is consistency. After you create and set up your environment, you can count on the desktop setup to perform in a certain way each time you use it, barring some sort of hardware or other error. When you're in a rush, you don't want to find that the resources you were counting on to get the job done suddenly become unavailable.

Working with an online IDE

Online IDEs have become popular for two reasons: They often cost nothing to use, and you can use them just about anywhere you have an Internet connection. This chapter discusses the use of Google

Colaboratory (often shortened to Colab) later, but you need to be aware that online IDEs exist in a lot of places and in many forms. Choosing the online IDE that you prefer based on your personal needs can require time, but after you've determined which one you want, you can load it up into any device that supports a browser and has a keyboard (even if that keyboard exists only on a screen).

USING A GAMING LAPTOP

Many people see a desktop system as a large, cumbersome box attached to a desk more or less permanently because it's too heavy to move. This perception is correct when you want to perform intense analysis tasks using huge datasets that reside in databases. But you can't use a standard laptop to perform heavy-duty analysis unless you choose an online IDE to do it. You can, however, use a high-end laptop, such as a gaming system, to perform many analysis tasks. For example, the HP Omen X (`https://www.amazon.com/exec/obidos/ASIN/B075LKSJZ7/datacservip0f-20/`) can work well for many needs. To get useful processing speed, the laptop system should have an Intel i7 processor and one of the GPUs listed at `https://developer.nvidia.com/cuda-gpus`. The GeForce GTX 1070 listed for the HP system actually ranks quite high on the compute capability listing at 6.1 (high-end GPUs top out at 7.5). Be aware that you need to plug the laptop in to AC power because the battery will never last long enough to perform a complex analysis. In addition, heat can become a problem in some situations, especially with a long analysis in warmer locations.

REMEMBER Even though online IDEs are normally associated with tablets and other smaller devices because of their connectivity flexibility, you can also use one with a desktop system. Of course, you use it within a browser, so you may not have the level of flexibility that you get with

a dedicated system. In addition, you likely won't make use of any special features possessed by your desktop system. This said, you can also get by with a much less expensive desktop system should you decide to go the online IDE route.

Considering the need for a GPU

A *CPU* is a general-purpose processor that can perform a wide assortment of tasks. The designers might not even have a clue about just what sorts of tasks the CPU design will perform. However, this general processing orientation takes space, so a CPU usually contains just a few cores — each of which can perform a single instruction at a time. An 8-core CPU can perform up to eight general tasks at a time.

GPUs perform a significantly more defined and less flexible array of tasks than CPUs, but this specialization has an advantage in size. Consequently, a GPU can perform a significantly greater number of tasks simultaneously because it typically has far more cores than a CPU does. The trade-off is that you can't build a complete application using a GPU. A GPU still requires input from a CPU to know what to do and when to do it. The following sections discuss GPUs in greater detail.

Using NVidia products for desktop systems

The reason NVidia appears so often in GPU discussions is that the company is at the forefront of making GPUs useful for tasks other than mere graphics display. Data science relies heavily on matrix manipulation (see Book 2, Chapter 3 for details about matrix manipulation), which is something a GPU does quite well. The company created the Compute Unified Device Architecture (CUDA), which makes it possible to use GPUs to perform data analysis. You can read a detailed discussion of precisely why CUDA is important at `https://www.datascience.com/blog/cpu-gpu-machine-learning`.

The point is that you can't use just any GPU for data science; you need a GPU with CUDA cores, in most cases.

REMEMBER Don't think that you get GPU support without a lot of effort. To use a GPU, you must install the required libraries and learn new programming techniques. Consequently, even though the processing speed is faster, you partly pay for it with greater development and testing time. Plus, applications that require a GPU to perform adequately limit the number of platforms on which they'll run.

TECHNICAL STUFF A GPU is more than just one order of magnitude more powerful than a CPU when it comes to performing specific math operations. Currently, the most powerful GPU in the world is the NVidia Titan V (`https://www.nvidia.com/en-us/titan/titan-v/`), which comes with 5,120 CUDA cores. That's 640 times the processing power of a CPU when you consider just the physical processing capacity. A GPU also optimizes math tasks, so the actual impact is significantly larger. The article at `https://www.anandtech.com/show/12673/titan-v-deep-learning-deep-dive` provides specifics on just how much more powerful a GPU is. However, to put this scenario into easily understood terms, a test application that required more than nine hours to run using just a standard PC with eight cores required only five minutes to run on a system sporting a Titan V GPU.

WARNING Note that adding a large GPU like the Titan V to your system will increase power and cooling requirements. You'll likely need additional cooling fans (which are noisy) to keep your system cool. Something that many people don't consider is that the addition could

make your office considerably warmer as well. If you don't address these additional requirements, you might find that your system can't complete processing tasks without overheating.

Defining framework needs

A framework, such as TensorFlow, which is used to perform deep learning tasks, creates an environment in which to run applications under controlled conditions. The "Presenting Frameworks" section, later in this chapter, describes precisely what a framework is and why you want to use one. However, the important thing to consider about frameworks and GPUs is that most frameworks provide some level of built-in GPU support. Consequently, you have all the advantages of using a GPU, with far fewer of the disadvantages.

Understanding the limits of online GPUs

When working with an online IDE, you may see an option to use a GPU with it, which is optimal when you can get it. However, you need to be aware that the GPU may not always be available, even if you check the option to request one. In addition, you can't be sure of what sort of GPU support you get, especially considering that the online IDE is likely free. The lack of GPU support is one of the things that makes using an online IDE risky if you need to process a lot of data and get the results quickly.

Choosing a Development Language

You might be amazed at the number of ways that people use computers and the tasks that computers perform without your knowledge. Computers come in all sorts of form factors — not just the smart phone, tablet, laptop, or desktop system you use. If you have newer appliances in your home, you might find that they contain computers. Even thermostats now sport computers that do everything from maintain complex operating schedules to report the efficiency of your sys-

tem in an email sent directly to your Inbox. So, it shouldn't surprise you that no single best computer language exists to express the myriad ideas that people have for using them in both mundane and interesting ways. In fact, it shouldn't even surprise you that people rely on a number of different languages to perform data analysis using data science techniques.

REMEMBER When contemplating the development language you want to use to write your data science application, you must consider the needs of your project and your personal needs first. A computer speaks only one language: machine code. In fact, the computer doesn't actually understand machine code; the machine code simply flips switches inside the processor to cause it to perform certain actions. The computer language you use always ends up translated into machine code at some point, so the language you use should meet your needs. The computer doesn't know or care about which language you use. With the goal of meeting your needs in mind, this book uses these two languages to help you perform data science tasks with greater ease:

- **Python:** Many data scientists choose Python because it's easy to learn, has strong community support, supports multiple programming paradigms, and provides access to a long list of useful packages. According to the article at `https://www.datanami.com/2018/07/19/python-gains-traction-among-data-scientists/`, a whopping 48 percent of younger data scientists prefer Python to any other language. These numbers go down for older data scientists.
- **R:** At one time, data scientists favored R over Python, but Python has caught up over the years, enough that you might wonder whether learning R is still a good idea. Statisticians originally created R to perform statistical analysis, and it's still the favored language for that purpose today because you can create incredibly complex models with it. Some developers also prefer the manner in which R helps you create presentations. In fact, the Shiny add-on (`http://shiny.rstudio.com/`) lets you create interactive applications directly from your R analysis. Yes, you have access to MatPlotLib in Python,

but R has built-in functionality that tends to work better for certain tasks. Like Python, R also supports a strong developer community that continually pumps out more packages to help you get work done faster. In some cases, developers choose R over Python simply because of the growing pains Python has suffered, as expressed by the article at `https://www.r-bloggers.com/why-r-for-data-science-and-not-python/`.

USING OTHER PROGRAMMING LANGUAGES FOR DATA SCIENCE

The reason this book uses these languages is that they represent popular choices for common data science and general application development needs, but you must remember that other languages support data science, too. One way to help you decide which language to choose when you have several in mind and they all seem equally good at the task you want to perform is to look at language popularity on sites such as Tiobe (`https://www.tiobe.com/tiobe-index/`). The Tiobe Index is one of the most cited language- popularity lists because it proves to be correct so often. (Alternatives include paid sites like IEEE Spectrum, where Python currently appears as the most used language.) However, make sure that you also consider factors such as your personal knowledge, availability of resources, cost, and so on when making a choice.

The article at `https://bigdata-madesimple.com/top-8-programming-languages-every-data-scientist-should-master-in-2019/` provides you with other language choices such as Java, SQL, and Julia. All these languages have benefits and potential problems. Choose your language carefully because many of them currently come with hidden issues and you may find that you can't make things work in the middle of a project because of a lack of library support. Most especially, make sure you look at issues such as plotting your data because some languages are weak in this area.

Obtaining and Using Python

If you choose to use Python to follow the book examples, you need to install a copy of Python on your system. The following sections provide detailed information for installing Python as part of Anaconda on a Windows system, with an overview for both Mac and Linux systems. If you don't plan to install Python on your system, you can skip this section.

Working with Python in this book

The Python environment changes constantly. As the Python community continues to improve Python, the language experiences *breaking changes* — those that create new behaviors while reducing backward compatibility. These changes might not be major, but they're a distraction that will reduce your ability to discover data science programming techniques. Obviously, you want to discover data science with as few distractions as possible, so having the correct environment is essential. Here is what you need to use Python with this book:

- Jupyter Notebook version 5.5.0
- Anaconda 3 environment version 5.2.0
- Python version 3.7.3

TIP If you don't have this setup, you may find that the examples don't work as intended. The screenshots will most likely differ and the procedures may not work as planned.

WARNING As you go through the book, you need to install various Python packages to make the code work. Like the Python environment you configure in this chapter, these packages have specific version numbers. If you use a different version of a package, the examples may not execute at all. In addition, you may become frustrated trying to work through error messages that have nothing to do with the

book's code but instead result from using the wrong version number. Make sure to exercise care when installing Anaconda, Jupyter Notebook, Python, and all the packages needed to make your deep learning experience as smooth as possible.

USE OF NOTEBOOK IN THE BOOK

This book shortens Jupyter Notebook to just Notebook in the interest of brevity. Whenever you see Notebook in a chapter, think of the Jupyter Notebook IDE.

Obtaining and installing Anaconda for Python

Before you can move forward, you need to obtain and install a copy of Anaconda. Yes, you can obtain and install Notebook separately, but then you lack various other applications that come with Anaconda, such as the Anaconda Prompt, which appears in various parts of the book. The best idea is to install Anaconda using the instructions that appear in the following sections for your particular platform (Linux, MacOS, or Windows).

Getting Continuum Analytics Anaconda

The basic Anaconda package is a free download from `https://repo.anaconda.com/archive/` to obtain the 5.2.0 version used in this book. Simply click one of the Python 3.6 Version links to obtain access to the free product. The filename you want begins with `Anaconda3-5.2.0-` followed by the platform and 32-bit or 64-bit version, such as `Anaconda3-5.2.0-Windows-x86_64.exe` for the Windows 64-bit version. Anaconda supports the following platforms:

- Windows 32-bit and 64-bit (The installer may offer you only the 64-bit or 32-bit version, depending on which version of Windows it detects.)
- Linux 32-bit and 64-bit
- macOS 64-bit

The free product is all you need for this book. However, when you look on the site, you see that many other add-on products are available. These products can help you create robust applications. For example, when you add Accelerate to the mix, you obtain the capability to perform multicore and GPU-enabled operations. The use of these add-on products is outside the scope of this book, but the Anaconda site provides details on using them.

Installing Anaconda for Windows

Anaconda comes with a graphical installation application for Windows, so getting a good install means using a wizard, much as you would for any other installation. Of course, you need a copy of the installation file before you begin, and you can find the required download information in the "Getting Continuum Analytics Anaconda" section, earlier in this chapter. The following procedure should work fine on any Windows system, whether you use the 32-bit or the 64-bit version of Anaconda:

1. **Locate the downloaded copy of Anaconda on your system.**
   The name of this file varies, but normally it appears as `Anaconda3-5.2.0-Windows-x86.exe` for 32-bit systems and `Anaconda3-5.2.0-Windows-x86_64.exe` for 64-bit systems. The version number is embedded as part of the filename. In this case, the filename refers to version 5.2.0, which is the version used for this book. If you use some other version, you may experience problems with the source code and need to make adjustments when working with it.
2. **Double-click the installation file.**
   (You may see an Open File – Security Warning dialog box that asks whether you want to run this file. Click Run if you see this dialog box pop up.) You see an Anaconda 5.2.0 Setup dialog box similar to the one shown in Figure 3-1. The exact dialog box you see depends on which version of the Anaconda installation program you download. If you have a 64-bit operating system, using the 64-bit version of Anaconda is always best for obtaining the best possible performance. This first dialog box tells you when you have the 64-bit version of the product.

FIGURE 3-1: The setup process begins by telling you whether you have the 64-bit version.

3. **Click Next.**

   The wizard displays a licensing agreement. Be sure to read through the licensing agreement so that you know the terms of usage.

4. **Click I Agree if you agree to the licensing agreement.**

   You're asked what sort of installation type to perform, as shown in Figure 3-2. In most cases, you want to install the product just for yourself. The exception is if you have multiple people using your system and they all need access to Anaconda. The selection of Just Me or All Users will affect the installation destination folder in the next step.

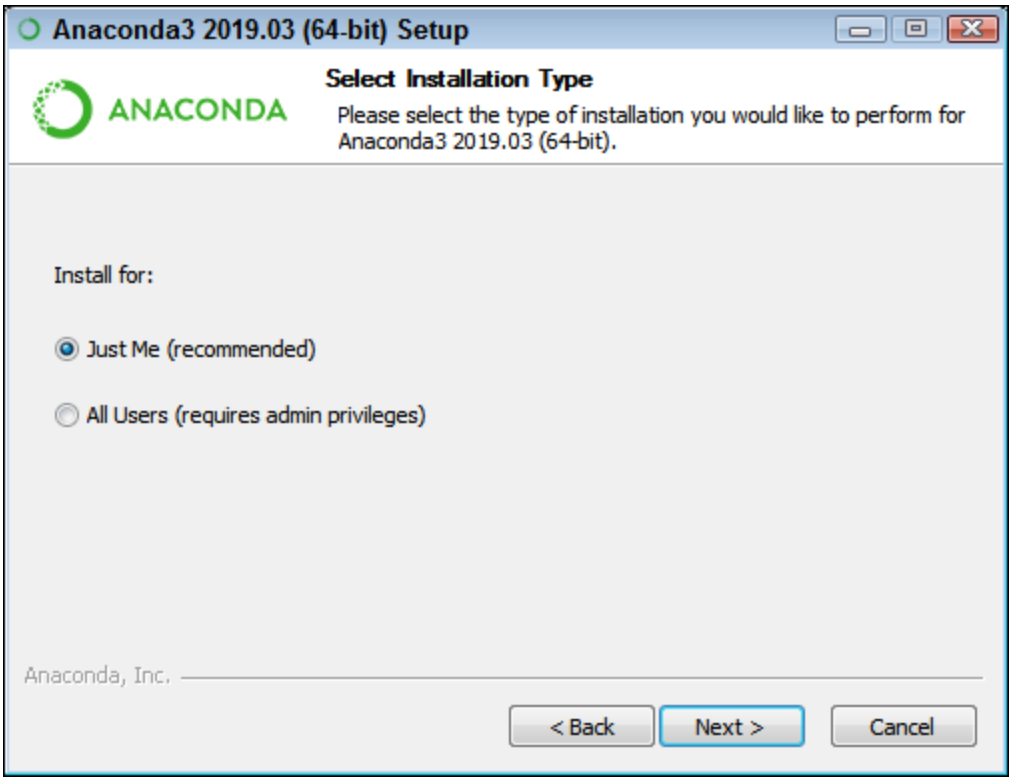

FIGURE 3-2: Tell the wizard how to install Anaconda on your system.

5. **Choose one of the installation types and then click Next.**

   The wizard asks where to install Anaconda on disk, as shown in Figure 3-3. The book assumes that you use the default location, which will generally install the product in your `C:\Users\ <User Name> \Anaconda3` folder. If you choose some other location, you may have to modify some procedures later in the book to work with your setup. You may be asked whether you want to create the destination folder. If so, simply allow the folder creation.

6. **Choose an installation location (if necessary) and then click Next.**

   You see the Advanced Installation Options, shown in Figure 3-4. These options are selected by default and you have no good reason to change them in most cases. You might need to change them if Anaconda won't provide your default Python 3.6 setup. However, the book assumes that you've set up Anaconda using the default options.

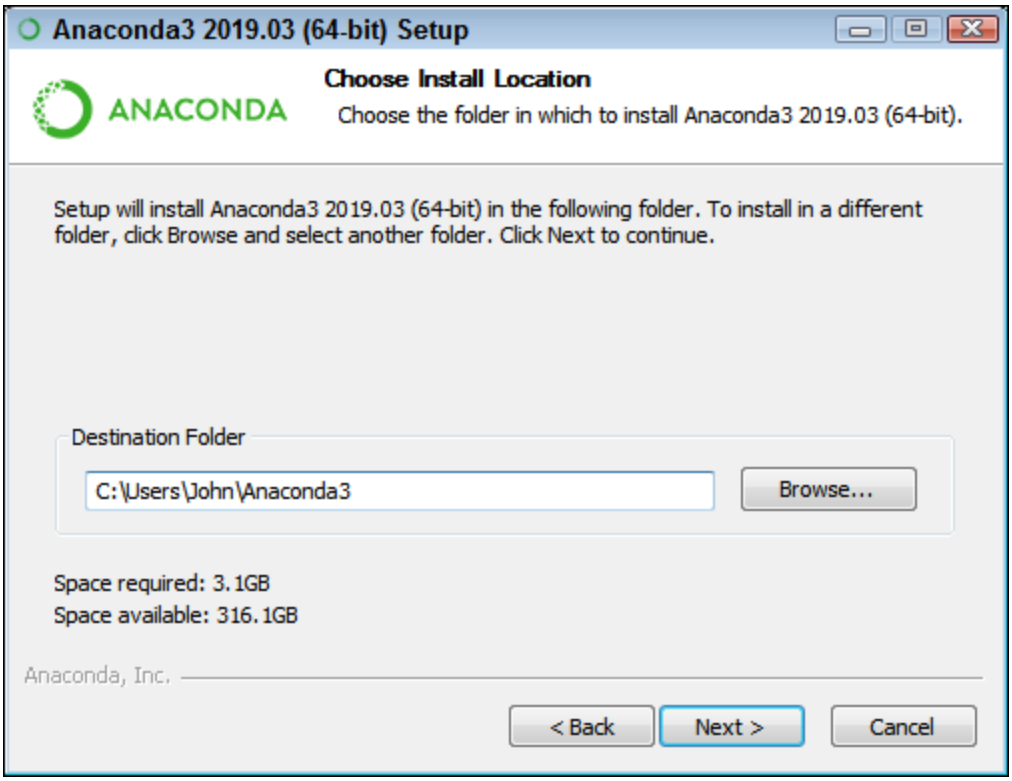

FIGURE 3-3: Specify an installation location.

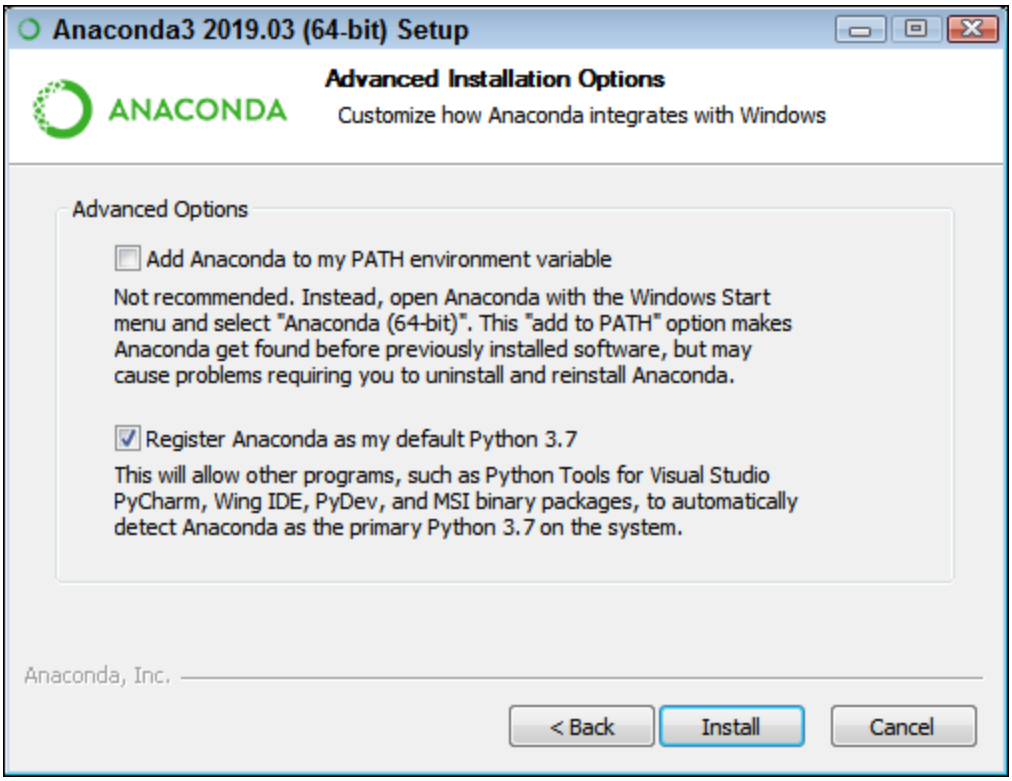

FIGURE 3-4: Configure the advanced installation options.

TIP The Add Anaconda to My PATH Environment Variable option is deselected by default, and you should leave it deselected. Adding it to the `PATH` environment variable does offer the ability to locate the Anaconda files when using a standard command prompt, but if you have multiple versions of Anaconda installed, only the first version that you installed is accessible. Opening an Anaconda Prompt instead is far better so that you gain access to the version you expect.

7. **Change the advanced installation options (if necessary) and then click Install.**
   You see an Installing dialog box with a progress bar. The installation process can take a few minutes, so get yourself a cup of coffee and read the comics for a while. When the installation process is over, you see a Next button enabled.
8. **Click Next.**
   The wizard tells you that the installation is complete.
9. **Click Next.**
   Anaconda offers you the chance to integrate Visual Studio code support. You don't need this support for this book, and adding it might change the way that the Anaconda tools work. Unless you absolutely need Visual Studio support, you want to keep the Anaconda environment pure.
10. **Click Skip.**
    You see a completion screen. This screen contains options to discover more about Anaconda Cloud and to obtain information about starting your first Anaconda project. Selecting these options (or deselecting them) depends on what you want to do next, and the options don't affect your Anaconda setup.
11. **Select any required options. Click Finish.**
    You're ready to begin using Anaconda.

Creating an Anaconda for Mac setup

As with the Windows installation, you must download a copy of Anaconda 5.2.0 for your Mac system. However, you can use a number of methods to perform this task based on the precise version of macOS that you own. The instructions at `https://mas-dse.github.io/startup/anaconda-macosx-install/` provide a terminal-based installation where you don't have to work through extra

steps to get the task done. This version relies on using curl to perform the download. The instructions at `https://docs.anaconda.com/anaconda/install/mac-os/` provide both a graphical installation and a terminal installation that works for other users. Choose the installation method that works best for your particular setup.

Creating an Anaconda for Linux setup

As with the Windows installation, you need a copy of Anaconda 5.2.0 to make the examples in this book execute properly. However, each version of Linux seems to come with slightly different installation instructions. Most Linux users will find that the instructions at `https://docs.anaconda.com/anaconda/install/linux/` or `https://docs.anaconda.com/anaconda/install/linux-power8/` work best. However, owners of an Ubuntu setup may find that they prefer the instructions at `https://www.osetc.com/en/how-to-install-anaconda-on-ubuntu-16-04-17-04-18-04.html` or `https://www.digitalocean.com/community/tutorials/how-to-install-anaconda-on-ubuntu-18-04-quickstart` better.

Defining a Python code repository

The code you create and use in this book will reside in a repository on your hard drive. Think of a *repository* as a kind of filing cabinet where you put your code. Notebook opens a drawer, takes out the folder, and shows the code to you. You can modify it, run individual examples within the folder, add new examples, and simply interact with your code in a natural manner. The following sections get you started with Notebook so that you can see how this whole repository concept works.

Defining the book's folder

You use folders to hold your code files for a particular project. The project for this book is DSPD (which stands for *Data Science Programming All-in-One For Dummies*). The following steps help you create a new folder for this book:

1. **Choose New⇒Folder.**
   Notebook creates a new folder for you. The name of the folder can vary, but for Windows users, it's simply listed as Untitled Folder. You may have to scroll down the list of available folders to find the folder in question.
2. **Select the box next to Untitled Folder.**
3. **Click Rename at the top of the page.**
   You see the Rename Directory dialog box, shown in Figure 3-5.
4. **Type** DSPD **and press Enter.**
   Notebook renames the folder for you.

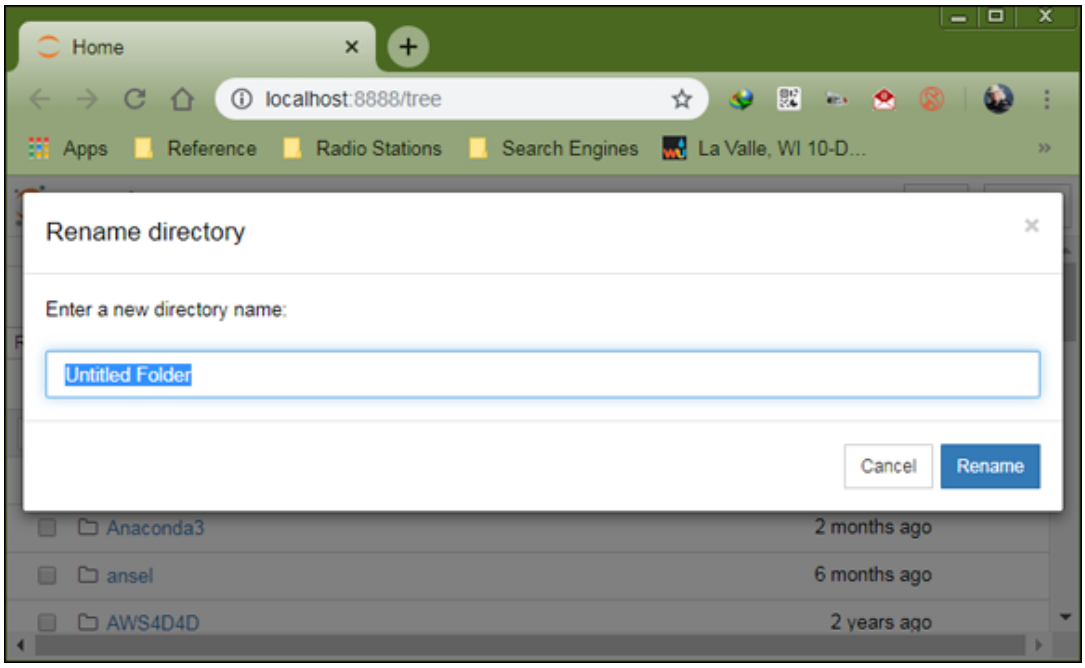

FIGURE 3-5: Create a folder to use to hold the book's code.

Creating a new notebook

Every new notebook is like a file folder. You can place individual examples within the file folder, just as you would sheets of paper into a physical file folder. Each example appears in a cell. You can put other sorts of data in the file folder, too, but you see how to perform these tasks as the book progresses. Use these steps to create a new notebook:

1. **Click the DSPD entry on the home page.**

   You see the contents of the project folder for this book, which will be blank if you're performing this exercise from scratch.

2. **Choose New⇒Python 3.**

   A new tab opens in the browser with the new notebook, as shown in Figure 3-6. Notice that the notebook contains a cell and that Notebook has highlighted the cell so that you can begin typing code in it. The title of the notebook is Untitled right now. That's not a particularly helpful title, so you need to change it.

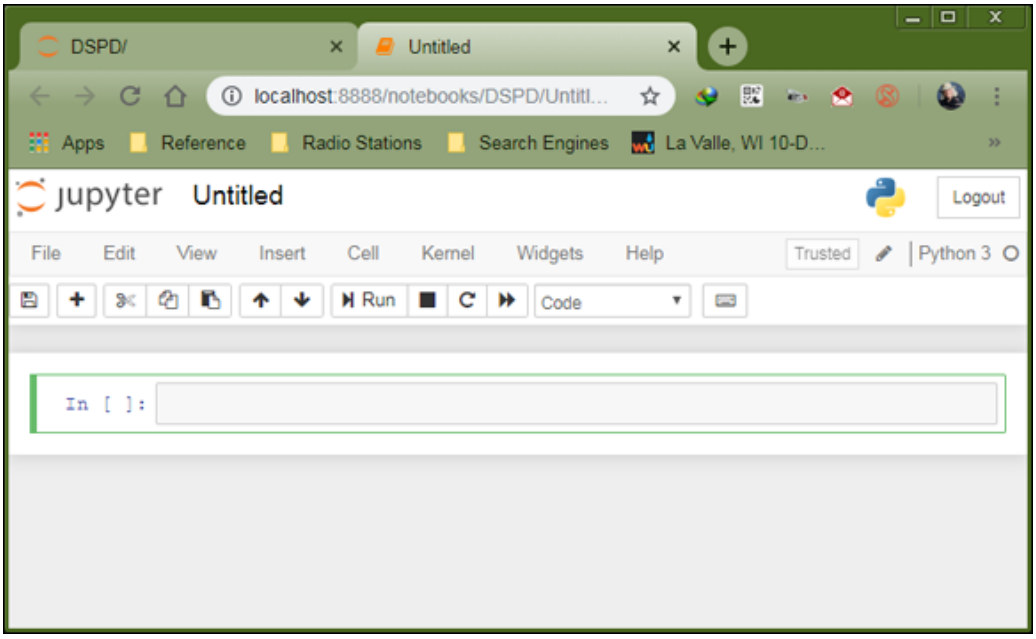

FIGURE 3-6: Provide a new name for your notebook.

3. **Click Untitled on the page.**

   Notebook asks what you want to use as a new name, as shown in Figure 3-7.

4. **Type** DSPD_0103_Sample **and press Enter.**

   The new name tells you that this is a file for *Data Science Programming All-in-One For Dummies,* Book 1, Chapter 3, `Sample.ipynb`. Using this naming convention helps you to easily differentiate these files from other files in your repository.

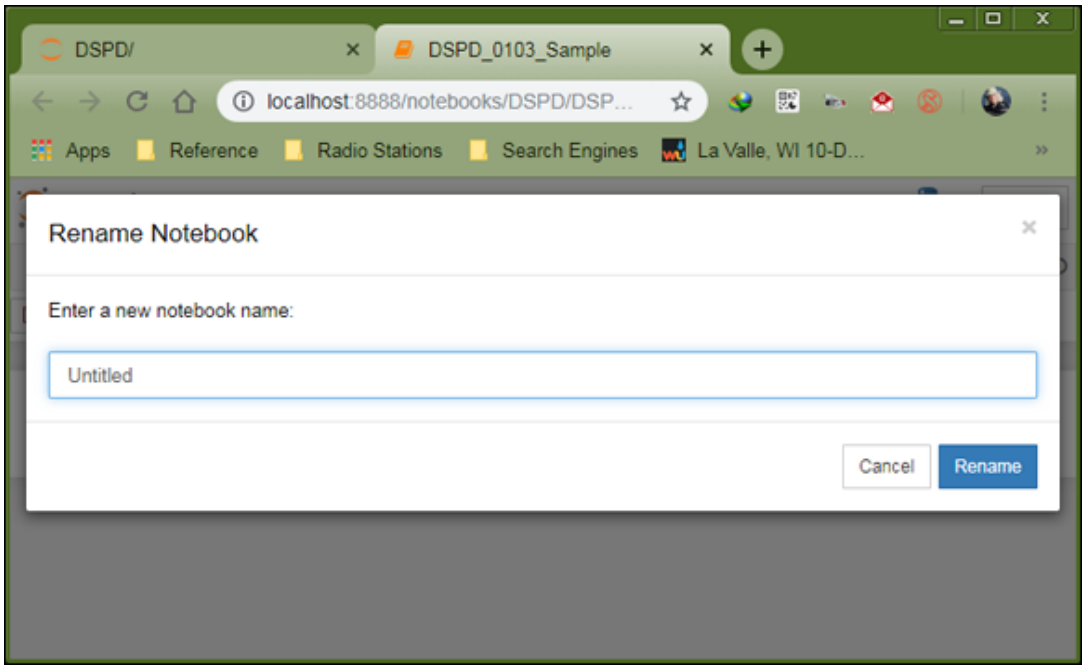

FIGURE 3-7: A notebook contains cells that you use to hold code.

After you create a new notebook, you see a cell that is ready to receive code. Simply type:

```
myString = "Hello, World!"
```

```
print(myString)
```

After you type the code, you can test it by clicking Run on the toolbar. Here is the output you can expect to see below the editing area, but within the same cell:

```
Hello, World!
```

TIP Notice that the code cell has a [1] next to it, showing that this is the first executed code cell in the Notebook. If you were to execute this cell again, the number would change to [2]. You can tell whether a cell is still executing because you see [*] instead of a number if it is.

Exporting a notebook

Creating notebooks and keeping them all to yourself isn't much fun. At some point, you want to share them with other people. To perform this task, you must export your notebook from the repository to a file. You can then send the file to someone else, who will import it into his or her repository.

The previous section, "Creating a new notebook," shows how to create a notebook named DSPD_0103_Sample. You can open this notebook by clicking its entry in the repository list. The file reopens so that you can see your code again. To export this code, choose File ⇒ Download As ⇒ Notebook (.ipynb). What you see next depends on your browser, but you generally see some sort of dialog box for saving the notebook as a file. Use the same method for saving the Notebook file as you use for any other file you save using your browser.

Saving a notebook

You eventually want to save your notebook so that you can review the code later and impress your friends by running it after you ensure that it doesn't contain any errors. Notebook periodically saves your notebook for you automatically. However, to save it manually, you choose File⇒Save and Checkpoint.

Closing a notebook

You definitely shouldn't just close the browser window when you finish working with your notebook. Doing so will likely cause data loss. You must perform an orderly closing of your file, which includes stopping the kernel used to run the code in the background. After you save your notebook, you can close it by choosing File⇒Close and Halt. You see your notebook entered in the list of notebooks for your project folder, as shown in Figure 3-8.

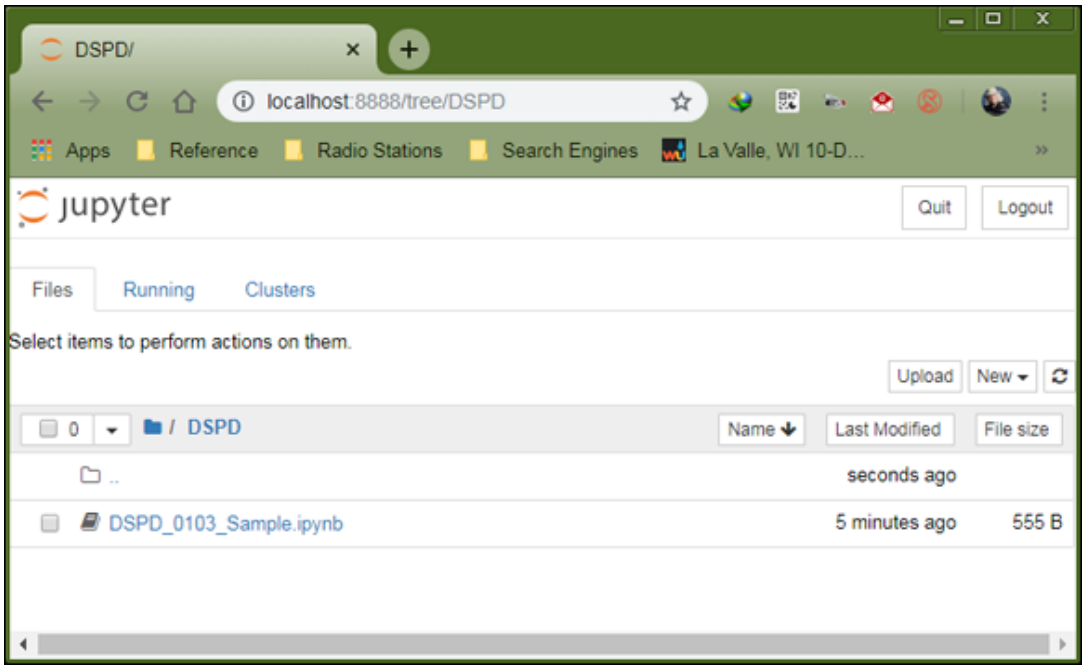

FIGURE 3-8: Your saved notebooks appear in a list in the project folder.

Removing a notebook

Sometimes notebooks get outdated or you simply don't need to work with them any longer. Rather than allow your repository to get clogged with files you don't need, you can remove these unwanted notebooks from the list. Use these steps to remove the file:

1. **Select the check box next to the DSPD_0103_Sample.ipynb entry.**
2. **Click the Delete (trash can) icon.**

   A Delete notebook warning message appears, like the one shown in Figure 3-9.
3. **Click Delete.**

   Notebook removes the notebook file from the list.

WARNING Exercise care when deleting notebook files. Notebook lacks any form of Undo for files, so trying to recover a deleted file can prove difficult.

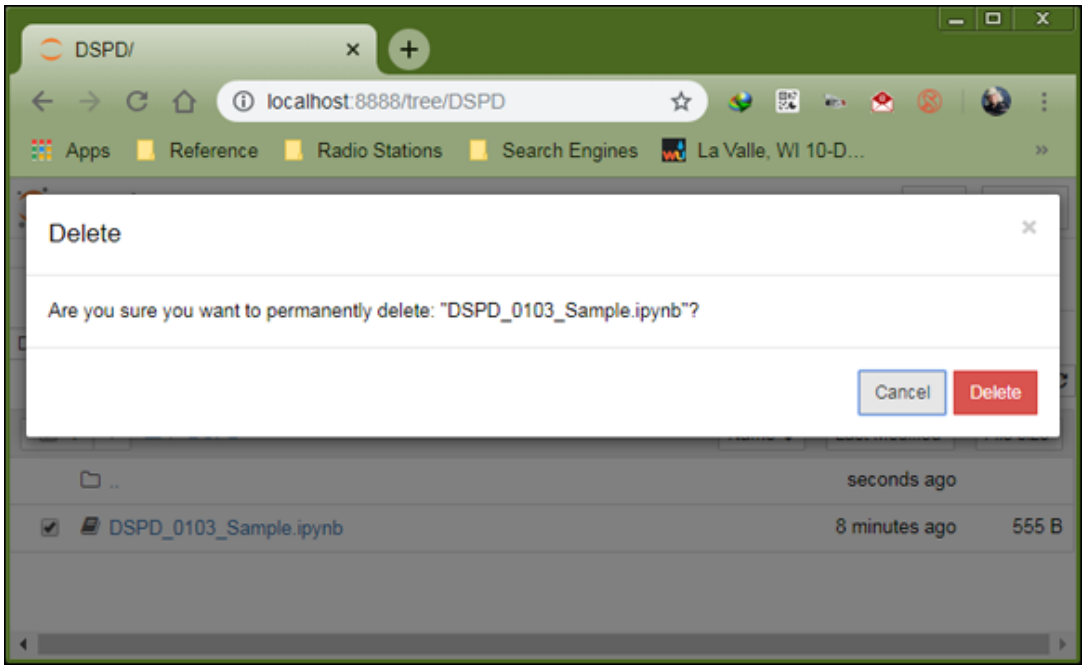

FIGURE 3-9: Notebook warns you before removing any files from the repository.

Importing a notebook

To use the source code from this book, you must import the downloaded files into your repository. The source code comes in an archive file that you extract to a location on your hard drive. The archive contains a list of `.ipynb` (IPython Notebook) files containing the source code for this book (see the Introduction for details on downloading the source code). The following steps tell how to import these files into your repository:

1. **Click the Upload on the Notebook DSPD page.**
   What you see depends on your browser. In most cases, you see some type of File Upload dialog box that provides access to the files on your hard drive.
2. **Navigate to the directory containing the files that you want to import into Notebook.**
3. **Highlight one or more files to import and then click the Open (or other, similar) button to begin the upload process.**
   You see the file added to an upload list, as shown in Figure 3-10. The file isn't part of the repository yet — you've simply selected it for upload.
4. **Click Upload.**
   Notebook places the file in the repository so that you can begin using it.

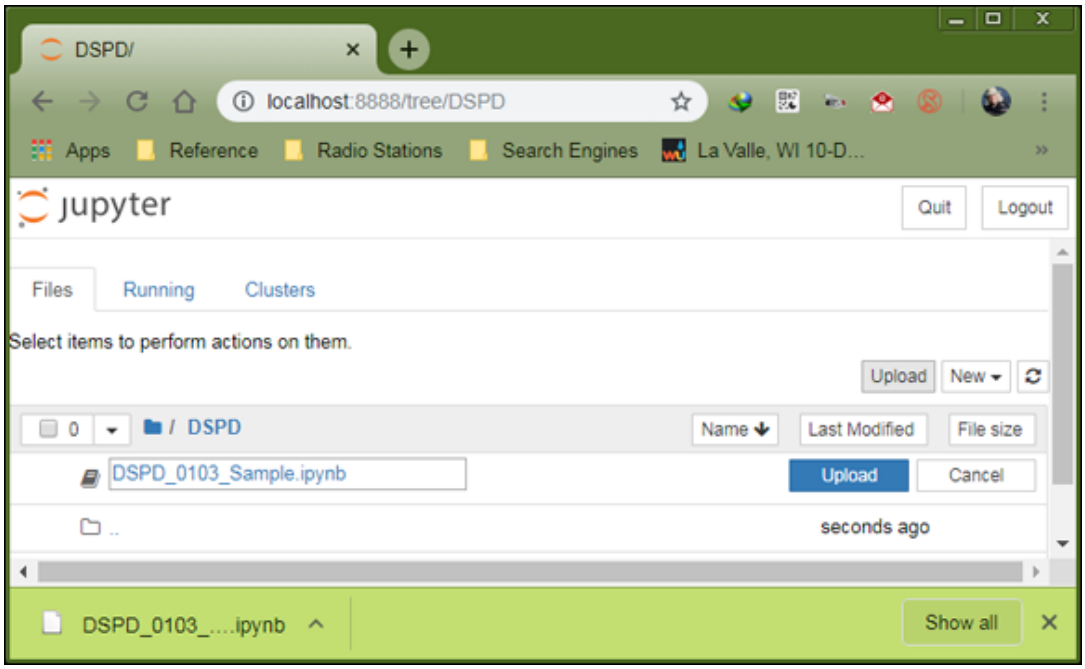

FIGURE 3-10: The files you want to add to the repository appear as part of an upload list.

Working with Python using Google Colaboratory

Colaboratory (`https://colab.research.google.com/notebooks/welcome.ipynb`), or Colab for short, is a Google cloud-based service that replicates Notebook in the cloud. This is a custom implementation, so you may find Colab and Notebook to be out of sync at times — meaning that features in one may not always work in the other. You don't have to install anything on your system to use Colab. In most respects, you use Colab as you would a desktop installation of Notebook. The main reason to learn more about Colab is if you want to use a device other than a standard desktop setup to work through the examples in this book. If you want a fuller tutorial of Colab, you can find one in Chapter 4 of *Python For Data Science For Dummies,* 2nd Edition, by John Paul Mueller and Luca Massaron (Wiley). For now, this section gives you the basics of using existing files. Figure 3-11 shows the opening Colab display.

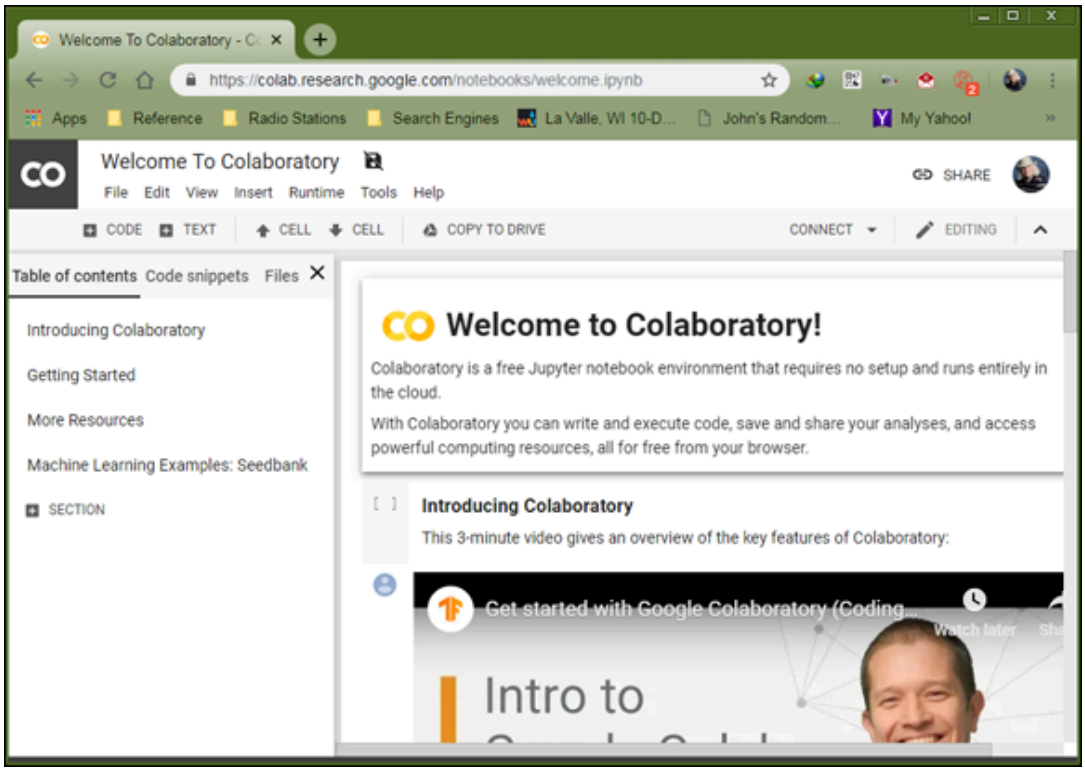

FIGURE 3-11: Colab makes using your Python projects on a tablet easy.

You can open existing notebooks found in local storage, on Google Drive, or on GitHub. You can also open any of the Colab examples or upload files from sources that you can access, such as a network drive on your system. In all cases, you begin by choosing File⇒Open Notebook. The default view shows all the files you opened recently, regardless of location. The files appear in alphabetical order. You can filter the number of items displayed by typing a string into Filter Notebooks. Across the top are other options for opening notebooks.

TIP Even if you're not logged in, you can still access the Colab example projects. These projects help you understand Colab but don't allow you to do anything with your own projects. Even so, you can still experiment with Colab without logging into Google first. Here is a quick list of the ways to use files with Colab:

- **Using Drive for existing notebooks:** Google Drive is the default location for many operations in Colab, and you can always choose it as a destination.

When working with Drive, you see a listing of files. To open a particular file, you click its link in the dialog box. The file opens in the current tab of your browser.

- **Using GitHub for existing notebooks:** When working with GitHub, you initially need to provide the location of the source code online. The location must point to a public project; you can't use Colab to access your private projects. After you make the connection to GitHub, you see a list of repositories (which are containers for code related to a particular project) and branches (which represent particular implementations of the code). Selecting a repository and branch displays a list of notebook files that you can load into Colab. Simply click the required link and it loads as if you were using Google Drive.
- **Using local storage for existing notebooks:** If you want to use the downloadable source for this book, or any local source, for that matter, you select the Upload tab of the dialog box. In the center, you see a single button called Choose File. Clicking this button opens the File Open dialog box for your browser. You locate the file you want to upload, just as you normally would for any file you want to open. Selecting a file and clicking Open uploads the file to Google Drive. If you make changes to the file, those changes appear on Google Drive, not on your local drive.

TIP Google is aware that people want to use Colab for their R projects (along with other languages; see the "Using other programming languages for data science" sidebar, earlier in this chapter, for details). However, Colab doesn't currently support these languages and no date set exists for implementing the required support. Consequently, if you choose to use Colab for the book's projects, you must work with Python.

Defining the limits of using Azure Notebooks with Python and R

As with Colab, you can use Azure Notebooks (see Figure 3-12) (`https://notebooks.azure.com/`) to run your Python code. Fortunately, in this case, an R kernel available, as described at

`https://notebooks.azure.com/help/jupyter-notebooks/available-kernels`. You can also install R packages and use them in your code, as described at `https://notebooks.azure.com/help/jupyter-notebooks/package-installation/r`. One of the most important issues that differentiate Colab and Azure Notebooks is that Google released Colab in 2014, so it has become a mature product, while Azure Notebooks is still in preview mode. Here are some other issues to consider:

- Colab supports 20GB of memory; Azure Notebooks is limited to 4GB.
- Azure Notebooks doesn't provide any GPU support.
- Azure Notebooks does provide better file support than Colab using the Libraries feature.
- Azure Notebooks provides configuration support using YAML files.
- Azure Notebooks provides an integrated bash terminal, but you can run bash commands directly in your Colab code using the `!` command.

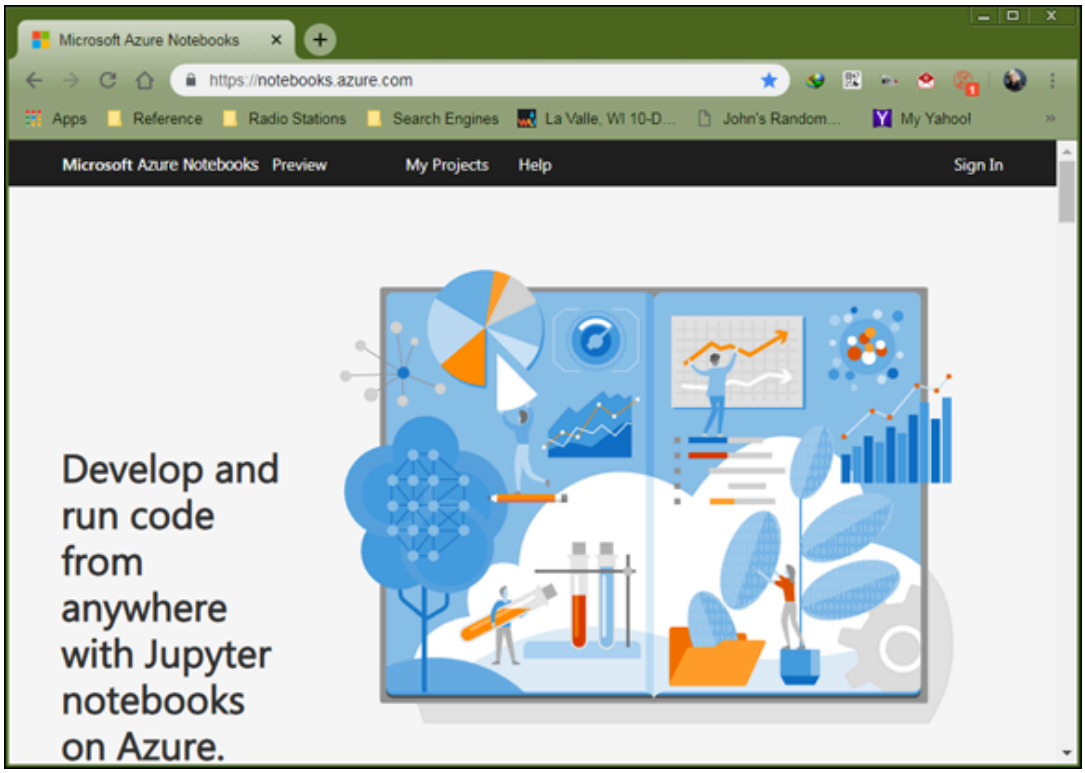

FIGURE 3-12: Azure Notebooks provides another means of running Python code.

Obtaining and Using R

This book relies on Anaconda for demonstrating the data science code. You must begin by installing Anaconda, as described in the "Obtaining and installing Anaconda for Python" section, earlier in this chapter. After you have an Anaconda installation in place, you can use the instructions in the following sections to create an R environment in which you can execute the book's R code.

Obtaining and installing Anaconda for R

To install R, you must open the Anaconda Prompt. For Windows users, this means choosing Start⇒ All Programs⇒ Anaconda3⇒ Anaconda Prompt. You see the Anaconda Prompt, shown in Figure 3-13. Notice the word *(base)* at the beginning of the prompt. This is the current environment — the base environment.

At the Anaconda Prompt, type **conda create -n R_env r-essentials r-base** and press Enter. This command creates a new Anaconda environment called R_env. Whenever you want to work with R code, you use the R_env environment. Within this environment, `conda`, the Anaconda command-line utility, installs R essentials and base packages. This set of packages is enough to get you started. However, you can install other packages later as needed for specific kinds of analysis. The installation process displays a series of messages that ends with a listing of packages that `conda` will install, like those shown in Figure 3-14.

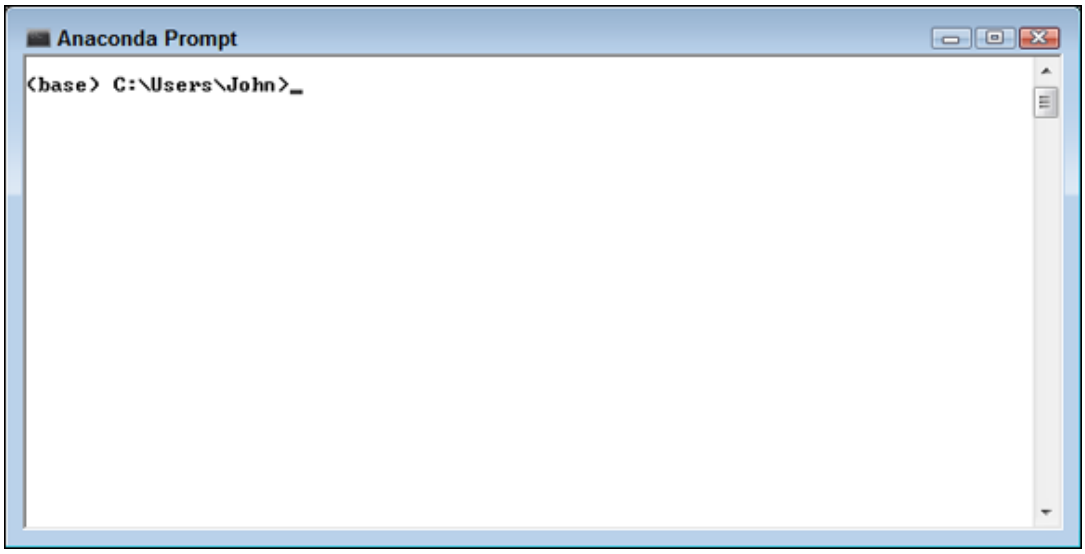

FIGURE 3-13: Open an Anaconda Prompt to install R.

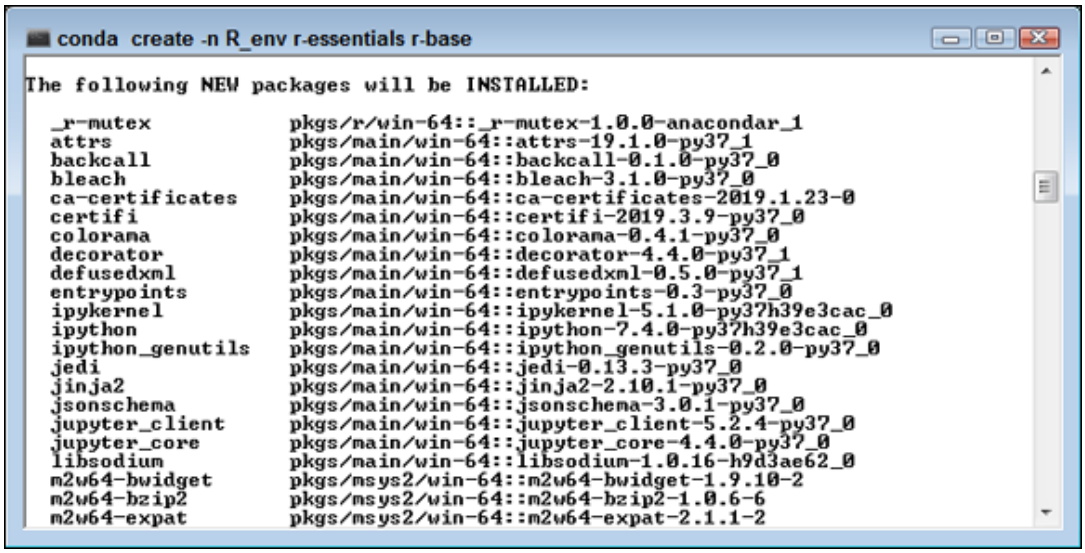

FIGURE 3-14: The conda utility tells you which packages it will install.

You can begin the installation process by typing **y** and pressing Enter. At this point, `conda` begins to download and extract packages. When the downloads complete, conda performs various transactional processes that can include compiling some of the packages it downloaded into executables (or other forms). This process can take a while, so you might want to have a good book ready or some coffee to drink.

REMEMBER After the installation process completes, you're still in the (base) environment. To work with the (R_env), you must type **conda activate R_env** at the Anaconda Prompt. You can how use whatever directory command your platform supports to see a list of the files that `conda` installed. To leave the (R_env) environment and go back to the (base) environment, you type **conda deactivate** and press Enter.

Starting the R environment

Fortunately, you won't work at the Anaconda Prompt when you want to work with R code. However, you won't start Notebook directly, either, because doing so starts the (base) environment and you want the (R_env) environment. Instead, you start Anaconda Navigator, which appears in Figure 3-15. Notice that this utility provides you with access to all the GUI tools that Anaconda supports (and if you don't see what you need, you can always install more).

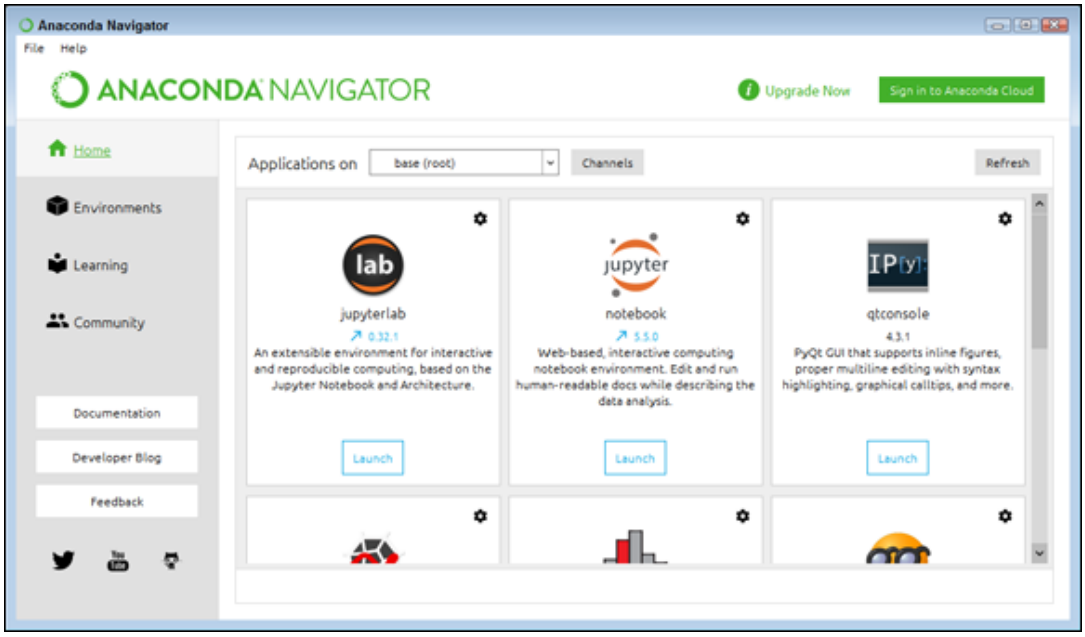

FIGURE 3-15: Anaconda Navigator provides access to a number of useful tools.

To start R, you must first select R_env in the Applications On drop-down list box, as shown in Figure 3-16. The tools you see will change

to reflect what you can do with this environment. The important tool for this book is Notebook. Note the gear icon in the upper-right corner of the square. If you need to change the version of Notebook, click the icon and choose Install Specific Version from the list.

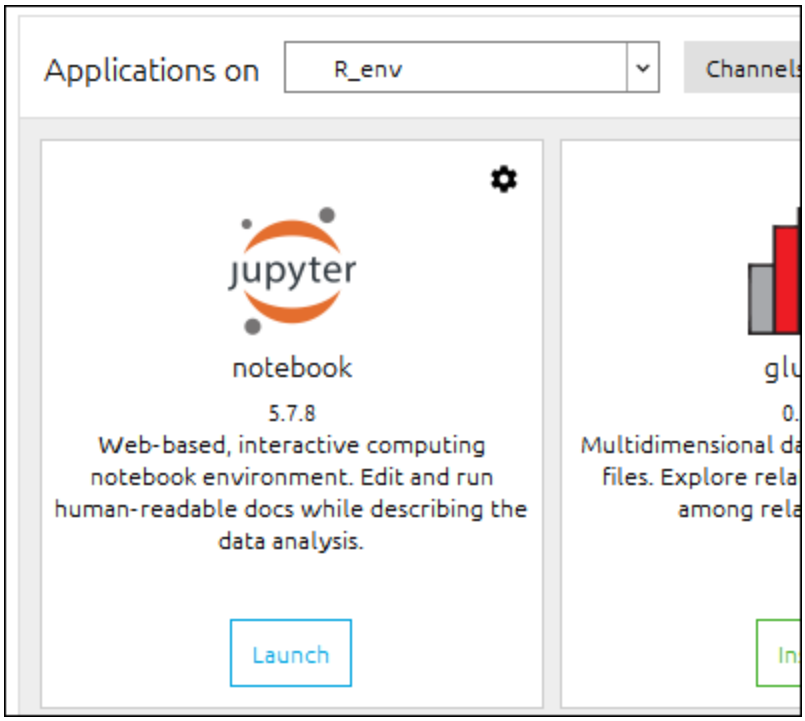

FIGURE 3-16: Changing your environment will often change the available tool list.

Click Launch in the Notebook square to start the application. You see Notebook start in a browser, just as you do when working with Python, but now you're working with R instead.

Defining an R code repository

Because you're using Notebook, everything with R works precisely the same as it does for Python, as described in the "Defining a Python code repository" section, earlier in this chapter. The name of the R repository for this book is `DSPD_R`. Use the instructions in the "Defining a Python code repository" section to create an R repository and experiment with an R file. However, instead of choosing Python 3 from the New drop-down list, you choose R instead. Otherwise, everything

works as it would for Python. The test file for this section is `DSPD_R_0103_Sample.ipynb`.

TIP Something important to note when using R is that you can download your code as an `.r` file, as shown in Figure 3-17. In fact, you have not only the Notebook formats to choose from, but a number of R-specific formats as well. To keep things simple, the downloadable source for this book relies on .ipynb files for all source code to ensure that you get the required comments with the file.

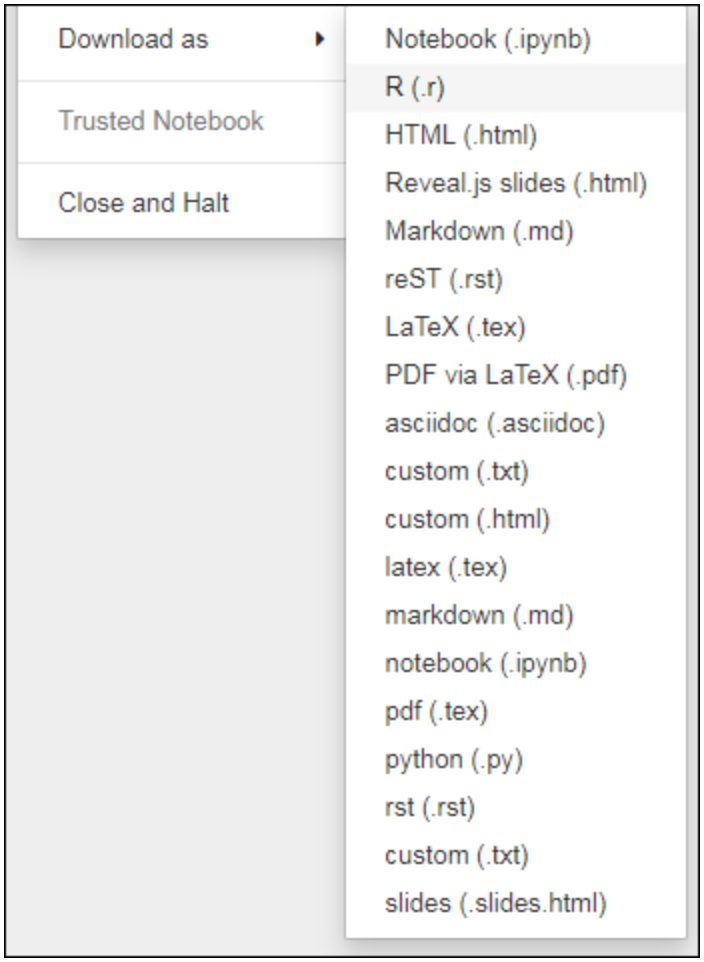

FIGURE 3-17: You can save R code in .r files, but the .r files lack Notebook comments.

You can perform a quick test of your R environment using the following code:

```
myString <- "Hello, World!"
```

```
print(myString)
```

After you click Run, you see the following output:

```
[1] "Hello, World!"
```

Presenting Frameworks

Sometimes you use a framework to work with an application, especially as you move from data science–specific analysis into other areas, such as machine learning and deep learning. Consequently, an overview of frameworks is helpful. You may find that you need to move from the environment provided for the majority of the examples in this book to something better suited for complex applications.

In a framework environment, your code makes requests of the framework, which then fulfills the request for you. Consequently, frameworks provide a kind of structure for application development. Because of this structure, frameworks are domain specific, answering specific kinds of application development needs. By taking care of some of the details for you and controlling the manner in which your application executes, a framework reduces the amount of coding you perform and makes your application both more reliable and more consistent.

The following sections discuss frameworks both from an overview perspective and in more detail as part of a machine learning or deep learning solution. It's important to remember that these sections don’t provide you with complete information on frameworks, but they do help you understand deep learning frameworks well enough to make good decisions about them.

Defining the differences

The problem domain–specific nature of frameworks makes it necessary to locate the right sort of framework for your needs. (A *problem domain* is a description of the expertise and resources required to solve a problem. For example, you wouldn't go to a doctor to solve your plumbing problems — you'd go to a plumber instead.) Simply asking for a general framework won't do you much good. Here are some examples of framework types, all of which have specific characteristics to meet the needs of their problem domain:

- Application framework (of the sort used to create end-user applications)
- Artistic (drawing, music, and other creative forms)
- Cactus framework (high-performance scientific computing)
- Decision support system
- Earth system modeling
- Financial modeling
- Web framework (including language-specific frameworks for languages like such as AJAX and JavaScript)

REMEMBER The diversity of software frameworks is amazing, and you're unlikely to ever need them all. They do have two important features in common. In each case, the framework defines a series of *frozen spots* that define the characteristics of the application and that the developer can't change. In addition, the framework defines *hot spots* that a developer does use to define the specifics to the target software. For example, a frozen spot in a web application might define the interface on which a user relies to make requests, while a hot spot might define how to fulfill that request. Someone designing a book search application would focus on the specifics of book searches while disregarding the requirements of state management and request handling.

Explaining the popularity of frameworks

In thinking about software, you can easily see the progression of tools used to create it. At one time, developers had to input their code using

keypunch cards, which was extremely time consuming and error prone. Editors made the job easier, allowing you to type what you want to get done. Next came the Integrated Development Environment (IDE). Using an IDE allows modeling, compilation, and testing of the code in a single environment, along with other things. The use of libraries enables you to create large, complex applications quickly. So, a framework — which is an environment in which a developer needs to consider only the specifies of a particular application — is simply the next step in making developers more productive while also making applications more robust and less error prone. Hence the popularity of frameworks with developers.

REMEMBER However, a framework is much more than simply a means of creating code faster, with less effort and fewer errors. A framework lets you create a standardized environment in which everyone uses the same libraries, tools, Application Programming Interfaces (APIs), and other programs. The use of a standardized environment enables you to transfer code between systems without fear of introducing odd application issues because of environmental inconsistencies. In addition, team development issues are fewer because the collaboration environment is simplified.

Because a framework handles all the low-level details, you must also consider the makeup of an application team. In the past, the team might need people who were adept at interacting with the hardware or creating user interface basics. The use of a framework means that all these tasks are already completed, so a team is made up of subject-matter experts who can communicate effectively with each other, making a coherent approach to application development possible.

The most important reason that frameworks are so popular now relates to how coding is done today. At one time, developers needed to know how to interact with the hardware and software at an extremely

low level. Today, frameworks make coding easy in an environment in which

- Most applications consist mainly of API calls strung together to achieve a specific purpose.
- People need to understand how APIs perform, rather than what they do or how they do it. A developer needs to consider what data structures the API accepts and how well it processes data under pressure.
- The immense installed base of existing software means keeping that code in place and finding fast, efficient methods to interact with it.
- The focus is on architecture rather than details. Because most new applications rely heavily on existing code accessed through libraries or APIs, developers don't spend as much time learning the idiosyncrasies of a language; it's better to discover which pile of code can do the work without having to write any of the code yourself.
- Getting the algorithm correct is what matters most.
- Tools have become so smart that they often correct minor coding errors and interpret ambiguities in developer code correctly, so the emphasis is on getting ideas down rather than writing perfect code.
- Visual languages, in which you drag and drop objects in a graphical environment, are becoming more common. At some point, code could actually disappear (at least, for most application developers).
- Knowing a single platform isn't enough. Most applications today must execute flawlessly on Windows, Linux, macOS, Android, most smart phones, and myriad other platforms because users want software in a form they understand.

CONSIDERING FRAMEWORK NEGATIVES

Depending on whom you talk to, a framework solution isn't always the panacea that supporters would make it out to be. One of the bigger issues when using a framework is that the framework becomes its own application. A development team needs to learn both the framework and all the tools used to write the application. Consequently, if most of the team members on a development effort haven't used the framework before, they'll need additional time to overcome the framework's learning curve. However, after they learn how to use a framework,

they'll easily gain back part of this initial investment in time through higher productivity overall.

Another problem with frameworks is their tendency to use resources inefficiently. The size of a framework application, framework included, is generally larger than an application developed using libraries. Of course, monolithic applications are generally the most efficient because they can use only the resources required for that application. All the code bloat found in frameworks comes from trying to create a one-size-fits-all solution.

The frameworks discussed in this book are all public offerings. In fact, most of them are open source as well. However, some proponents of frameworks feel that every enterprise should have its own framework that is developed using the common code from applications in that enterprise. With that approach, the resulting framework has a consistent look and feel that matches the pre-framework applications that the enterprise has to maintain. However, developing a custom framework for a particular enterprise is time consuming. Therefore, many people point out that a framework-based solution isn't as useful or easy to learn as nonframework solutions.

Choosing a particular library

The previous sections in this chapter discuss the appeal of frameworks in general and trace how frameworks can create a significantly better work environment for developers. Also covered are features that make a framework used for machine learning or deep learning special. Of course, the amount of automation that a framework supplies and the number of typical features it supports are the starting point for finding a framework that meets your needs. You also need to consider issues such as learning curve with regard to the ease of using the framework.

TIP One of the more important considerations when choosing a framework is to remember that frameworks are domain specific, which means that if you need to create an application that spans domains, such as a deep learning application that includes a web interface, you need multiple frameworks. Getting frameworks that work well with each other can be critical. If you also host your application in the cloud, you need to consider which frameworks work with the cloud vendor's offering, too. For example, if you choose to use TensorFlow as your framework, you can also rely on Amazon Web Services (AWS) to host your application (see `https://aws.amazon.com/tensorflow/` for details).

REMEMBER As another option when using TensorFlow, you can go directly to Google Cloud (see `https://cloud.google.com/tpu/` for details), where you can train your deep learning solution using GPUs or Tensor Processing Units (TPUs). The TPUs were developed by Google specifically for neural network machine learning use TensorFlow. TPUs are Application-Specific Integrated Circuits (ASICs) optimized for a particular use. In this case, they're for neural network processing using TensorFlow.

Application size and complexity also play a role in deep learning framework choice because you often need a higher-end framework to interact properly with large applications. The need to deal with applications of various sorts is offset by the usual cost and availability concerns. Many of the low-end deep learning frameworks in this chapter cost you nothing to try and could provide everything needed to get started.

Accessing the Downloadable Code

The "Importing a notebook" section of this chapter discusses the mechanics of importing an `.ipynb` file into the Notebook environment. Each of the languages used for this book has its own separate folder in the downloadable source: Python and R. To access the downloadable code for a particular example, you first locate the correct language file. Inside the folder, you find the source code files that you can then import into Notebook.

REMEMBER Some of the book examples come with prerequisites. You might have to load a new library or perform a configuration change. The downloadable code won't work properly without these changes. As part of performing any setups, you need to ensure that you install the correct versions of any libraries or packages. Otherwise, breaking changes in older or newer versions of these elements will cause the code to fail in some extremely odd ways. Trying to figure out just why a particular piece of code is failing in this circumstance, especially through email, can prove nearly impossible.

Even though this chapter discusses the use of online products to execute the code, these options come without any sort of guarantee, and you need to know how to upload the downloadable source to the online product you want to use. In some cases, all you really need to do is drag and drop the file from where you downloaded it to the online product.

Table of contents collapsed